PROGRAMACIÓN EN MÓVILES

LABORATORIO 11-12

Compose + MVVM + Retrofit + Room + Firebase Auth

TecsupNite: la tienda de Fortnite de hoy

Alumno: Renzo Raúl León Fernández

Docente: Juan José León S.

Curso: Programación en Móviles, 4to ciclo, Diseño y Desarrollo de Software (C24), sección D

Modalidad: individual

Fecha: 11 de noviembre de 2026

1. Datos del alumno y del proyecto

Dato	Valor
Alumno	Renzo Raúl León Fernández
Proyecto Android	TecsupNite, paquete com.tecsup.fortnite
Proyecto Firebase	tecsupnite (Authentication con correo y contraseña, y Cloud Firestore)
Repositorio GitHub	https://github.com/RXDG2908/Moviles-Android-CD
Carpeta del proyecto	Semana 12 / TecsupNite
Ramas	main con la Fase 1 y mejora-ia con la Fase 2 hecha con IA
Explicación del proyecto	README.md y PROMPTS.md dentro de la carpeta del proyecto

2. Objetivo

Construir una aplicación Android completa con el patrón MVVM que junte las cuatro tecnologías vistas en teoría: Jetpack Compose con Material 3 para la interfaz, Retrofit para leer un servicio web, Room para guardar los datos en el celular y Firebase para el ingreso con dos tipos de usuario, además de una notificación local al terminar de sincronizar.

Tema y variante

El docente permitió usar cualquier API pública que cumpla con lo que pide la guía. Elegí la API de Fortnite, que no necesita clave. La lista sale de la tienda del día y el detalle de cada cosmético se pide con un segundo servicio. Como variante, la app muestra solo cosméticos de Battle Royale: se dejan fuera las pistas musicales, los instrumentos y los autos, y los packs aparecen como lote con el precio del pack completo.

Uso	Servicio
Lista	Tienda de hoy de la API de Fortnite, en español
Detalle	Cosmético por su código, de la misma API

3. Configuración del proyecto en Firebase (Google Cloud)

Firebase es un servicio de Google Cloud que da el ingreso de usuarios y la base de datos en la nube sin tener que crear un servidor propio. Para este laboratorio creé el proyecto tecsupnite y lo configuré con los pasos de la guía:

1. Crear el proyecto en la consola de Firebase con el nombre TecsupNite, sin Google Analytics.
2. Registrar la app Android con el mismo paquete que la aplicación, com.tecsup.fortnite, y descargar el archivo de configuración dentro de la carpeta de la app. Ese archivo no se sube a GitHub porque tiene la clave del proyecto.
3. En Authentication, activar el ingreso con correo electrónico y contraseña.
4. Crear dos usuarios de prueba, uno administrador y otro normal.
5. Crear la base de datos de Cloud Firestore y la colección de usuarios, con un documento por usuario cuyo código es el mismo que tiene en Authentication, y los campos rol y nombre.
6. Publicar las reglas de seguridad: cada usuario solo puede leer su propio documento y nadie puede escribir desde la app.

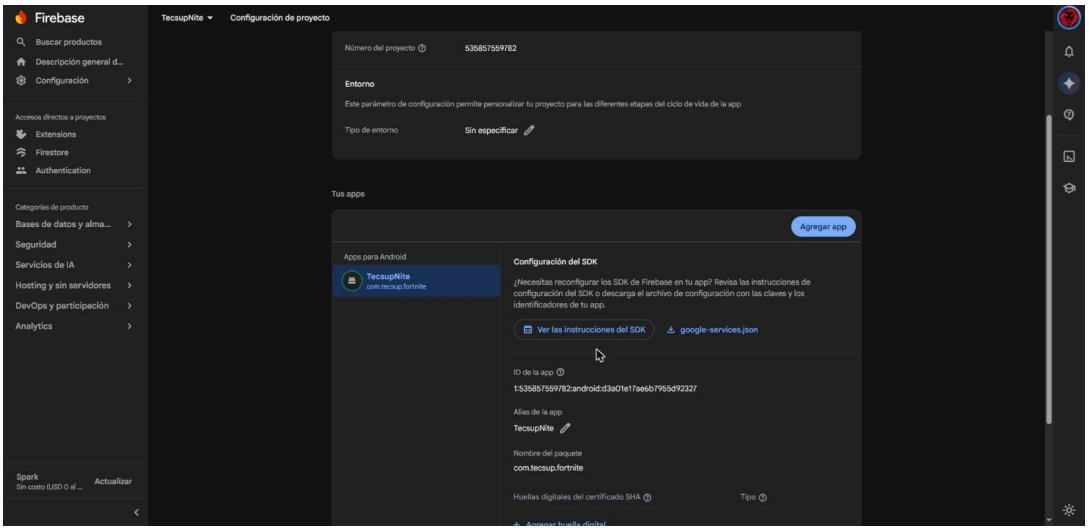


Figura 1. Configuración del proyecto con la app Android TecsupNite registrada con el paquete com.tecsup.fortnite.

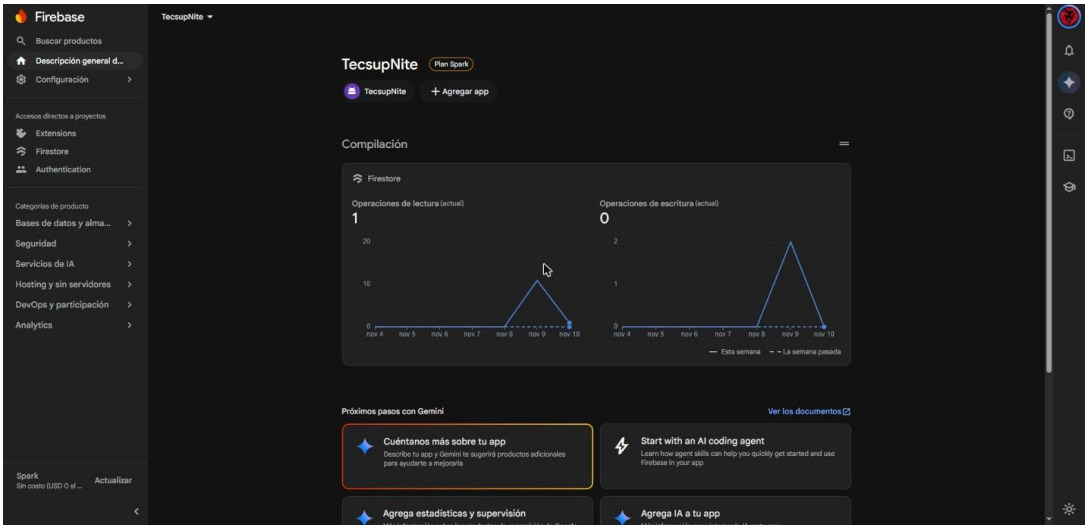


Figura 2. Descripción general del proyecto TecsupNite con las lecturas y escrituras de Firestore.

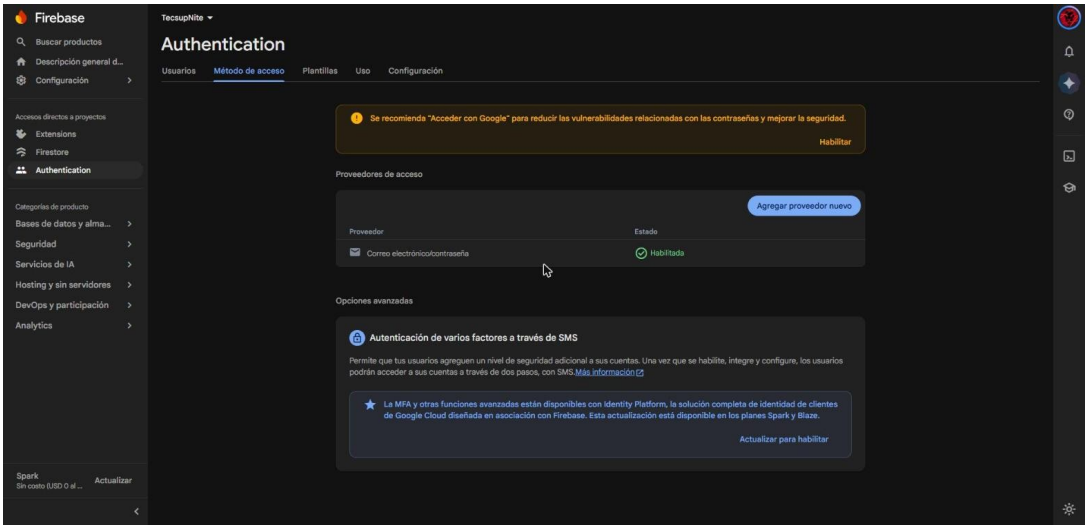


Figura 3. Authentication con el método de acceso por correo electrónico y contraseña habilitado.

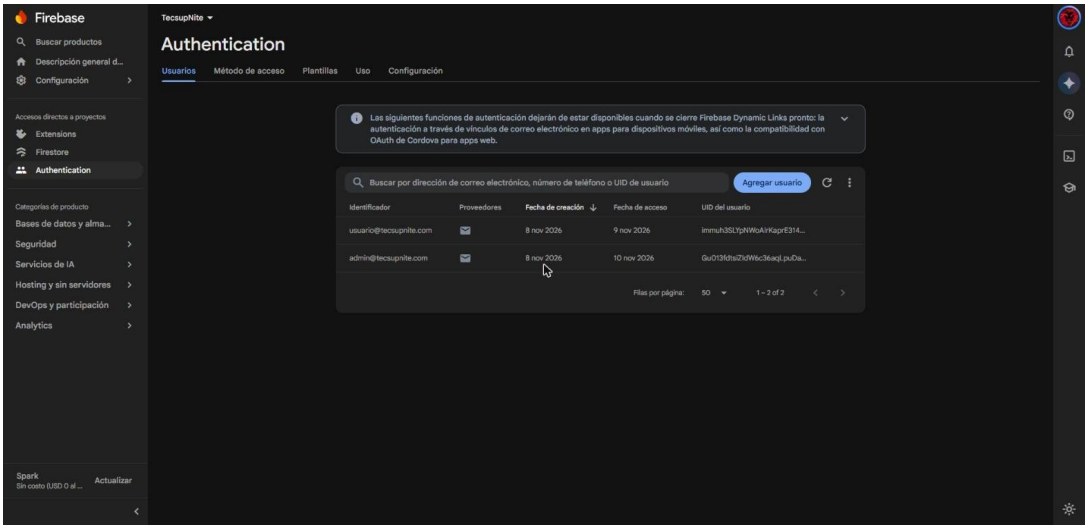


Figura 4. Los dos usuarios de prueba: administrador y usuario normal.

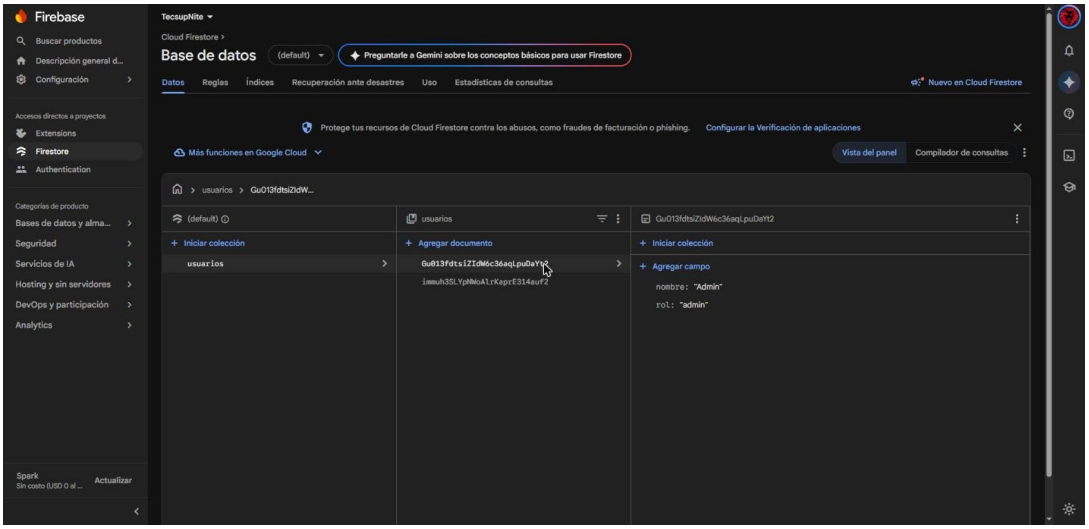


Figura 5. Colección de usuarios en Firestore: el documento del administrador con rol admin.

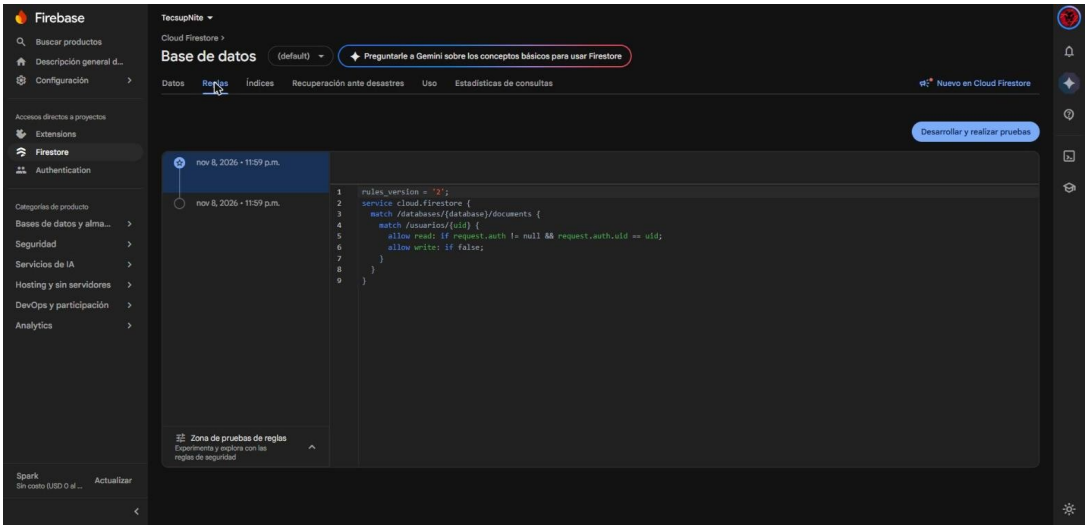


Figura 6. Reglas de seguridad publicadas en Firestore.

4. Requisitos de la guía

La tabla resume cómo cumple la app cada requisito. Los niveles son los de la guía: esencial, guiado y avanzado.

Código	Requisito	Cómo se cumple
R01	Login con correo y contraseña	La pantalla de ingreso revisa que los campos no estén vacíos y que el correo tenga un formato válido antes de llamar a Firebase. Si la contraseña es incorrecta muestra un mensaje claro y no deja entrar.
R02	Dos roles guardados en Firestore	El rol no se decide en la app: después de ingresar se lee el documento del usuario en la colección de usuarios de Firestore. Solo el administrador ve la pestaña del panel.
R03	Lista con estados	La tienda muestra un indicador mientras carga, una pantalla de error con el botón Reintentar si algo falla y un mensaje cuando no hay cosméticos.
R04	Detalle con el segundo servicio	Al tocar un cosmético se abre una hoja con su imagen, rareza, descripción, serie y en qué temporada llegó, pedidos al servicio de detalle.
R05	Arquitectura MVVM	Las pantallas solo observan el estado que exponen los ViewModel. Cada ViewModel depende de la interfaz del repositorio, no de la clase concreta, y se crea con una fábrica que recibe el contenedor de la app.
R06	Room	Lo descargado se guarda en una base local con su tabla, su acceso a datos y su base de datos. La lista que ve el usuario sale siempre de Room y el servicio web solo la actualiza.
R07	Favoritos	Cada tarjeta tiene un botón para marcar o quitar el favorito. Los favoritos tienen su propia pantalla y se conservan al cerrar la app y al volver a sincronizar.
R08	Modo sin conexión	Sin internet se muestra lo guardado con un aviso. Si no hay nada guardado aparece el error con Reintentar.
R09	Notificación local	Al terminar de sincronizar llega el aviso de que se guardaron los elementos. Usa un canal propio, pide el permiso de notificaciones y al tocarla se abre la app.
R10	Panel Admin	Muestra el total guardado y el total de favoritos, y tiene los botones para sincronizar y para limpiar la caché.
R11	Sesión	La sesión se mantiene al reabrir la app y hay un botón para cerrar sesión.
R12	Pruebas del ViewModel	En la rama de la Fase 2 hay seis pruebas del ViewModel de la lista con un repositorio falso en memoria, y nueve más para la búsqueda y el login.
R13	Mejora con IA documentada	Las mejoras de la Fase 2 están en la rama mejora-ia y cada consulta a la IA está explicada en el archivo de prompts.

5. Arquitectura del proyecto

El proyecto respeta la estructura de carpetas obligatoria de la guía. Cada capa tiene una sola responsabilidad:

- **Datos remotos:** la interfaz del servicio web, el cliente de Retrofit y las clases que copian la respuesta de la API.
- **Datos locales:** la tabla de Room, sus consultas y la base de datos.
- **Ingreso:** la interfaz de autenticación y su versión con Firebase, que además traduce los errores de Firebase a motivos propios.
- **Repositorio:** una interfaz y su implementación, que descarga la tienda, la guarda en Room y conserva los favoritos.
- **Conversión:** tres modelos distintos, uno para la API, otro para Room y otro para la pantalla, con funciones que pasan de uno a otro.
- **Notificación:** una interfaz para avisar y su versión con las notificaciones de Android, para que el ViewModel no dependa del sistema.
- **Interfaz:** las pantallas de ingreso, tienda, detalle, favoritos y panel, cada una con su ViewModel, más la navegación y el tema.

Diseño

El diseño se basa en mi tienda web regalitos.shop: fondo azul en degradado, paneles azul oscuro con borde celeste, amarillo como único color de acción y la fuente Burbank del juego solo en los títulos, con Inter para el resto del texto. La tienda sigue el mismo orden que la web: cabecera con la cuenta regresiva hasta el cambio de tienda, búsqueda, filtros con su cantidad y las secciones reales de la tienda del día.

6. Capturas de la aplicación

Capturas tomadas en el emulador con la tienda real y la cuenta de administrador.



Figura 7. Pantalla de ingreso.

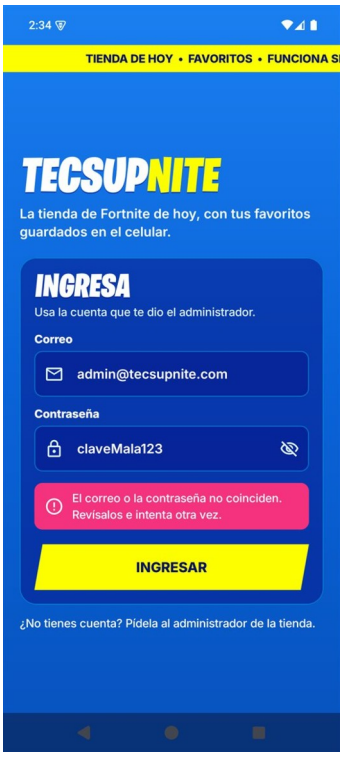


Figura 8. Ingreso con contraseña incorrecta.

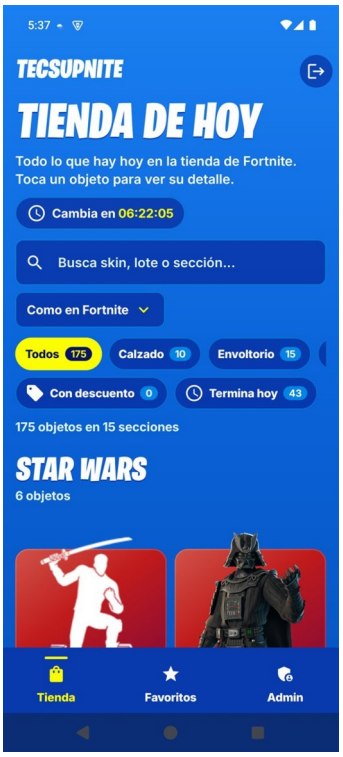


Figura 9. Tienda de hoy con búsqueda y filtros.

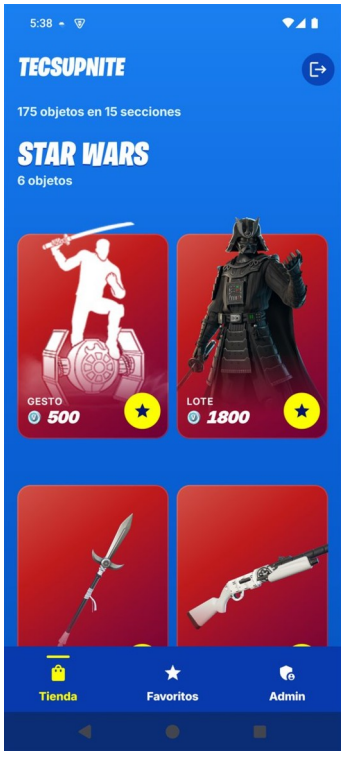


Figura 10. Sección de la tienda con favoritos marcados.



Figura 11. Hoja con el detalle del cosmético.

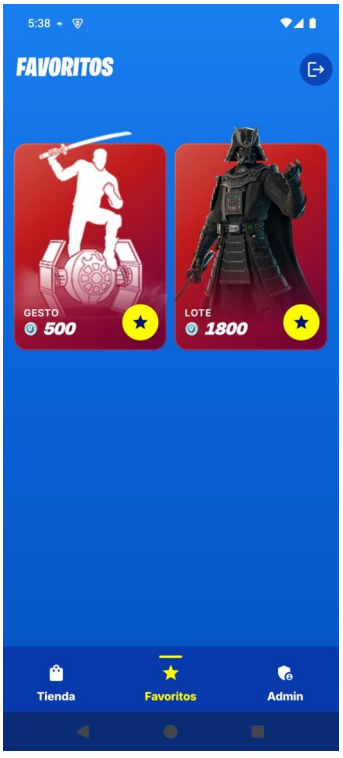


Figura 12. Pantalla de favoritos.

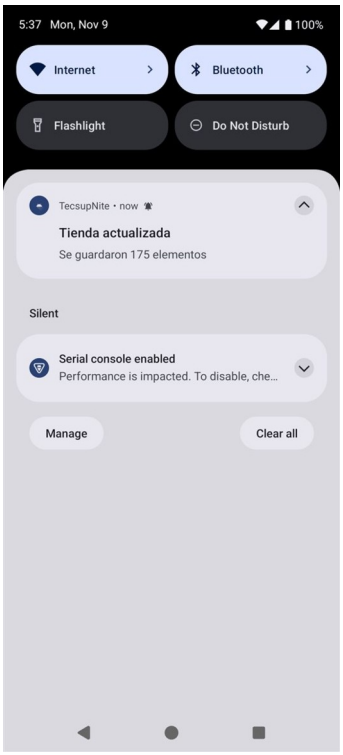


Figura 13. Notificación al terminar de sincronizar.



Figura 14. Panel del administrador.

7. Pruebas de aceptación

Las catorce pruebas de la guía se hicieron en el emulador con la tienda real y Firebase real.

Código	Caso	Resultado
PA-01	Ingresar con el usuario normal	Entra a la tienda y no ve el panel del administrador.
PA-02	Ingresar con el administrador	Entra y ve la pestaña del panel.
PA-03	Contraseña incorrecta	Mensaje claro y no entra.
PA-04	Correo o clave vacíos	Muestra el error debajo de cada campo y no llama a Firebase.
PA-05	Primera apertura con internet	Se ve la carga, luego la lista y llega la notificación.
PA-06	Tocar un elemento	Se abre el detalle con los datos del segundo servicio.
PA-07	Marcar un favorito y cerrar la app	Al reabrir sigue en favoritos.
PA-08	Modo avión con datos guardados	Se ve la lista con el aviso de sin conexión.
PA-09	Modo avión sin datos	Pantalla de error con el botón Reintentar.
PA-10	Sincronizar después de marcar un favorito	El favorito no se pierde.
PA-11	El administrador limpia la caché	Los totales quedan en cero y al sincronizar vuelven los datos.
PA-12	Cerrar sesión y reabrir la app	Vuelve al ingreso y pide las credenciales.
PA-13	Girar la pantalla	No se pierde el estado y la tienda usa más columnas.
PA-14	Negar el permiso de notificaciones	La app funciona normal, sin avisos.

8. GitHub y Fase 2

Ramas y commits

- La rama main tiene la Fase 1 con más de quince commits pequeños, cada uno con su tipo y un mensaje claro en español.
- La rama mejora-ia tiene la Fase 2 con más de tres commits y el archivo de prompts.
- No se suben las carpetas de compilación, la configuración local ni el archivo de Firebase con la clave de la API.
- La Fase 2 se entrega en su propia rama, sin Pull Request, porque main y mejora-ia tienen historias separadas desde los laboratorios anteriores.

Mejoras de la Fase 2

Mejora	Qué hace
Pruebas unitarias	Prueban el ViewModel de la lista con un repositorio y un notificador falsos: sincronizar, error sin datos, sin conexión con datos, favoritos, deslizar para actualizar y reintentar.
Búsqueda sin tildes	Encuentra un cosmético aunque se escriba sin tildes ni mayúsculas, y busca por nombre, sección y tipo.
Deslizar para actualizar	Al bajar la tienda con el dedo se sincroniza sin quitar la lista de la pantalla.
Login más claro	Etiquetas arriba de cada campo, botón para ver la contraseña, error debajo de cada campo y mensajes distintos para contraseña incorrecta, falta de internet y muchos intentos.

Resultado de las pruebas unitarias: quince pruebas en total, todas aprobadas.

Grupo de pruebas	Pruebas	Fallas
ViewModel de la lista	6	0
Búsqueda	4	0
ViewModel del login	5	0

9. Preguntas de la sesión

Respuestas a las preguntas sobre notificaciones de la teoría de la semana.

1. Gimnasio que recuerda entrenar a las 7 p. m.

Usaría una notificación local. La hora la conoce la misma app y el recordatorio es igual todos los días, así que el celular puede programarlo solo, sin internet y sin un servidor.

2. Tienda online que avisa que un producto fue enviado

Usaría una notificación push. El envío ocurre en el sistema de la tienda, fuera del celular, y el aviso tiene que llegar aunque la app esté cerrada, así que lo manda el servidor.

3. Banco que informa una transferencia

Usaría una notificación push. La transferencia se registra en el servidor del banco y el cliente debe enterarse en el momento, por seguridad, aunque no tenga la app abierta.

4. App de tareas con un recordatorio cada mañana

La más adecuada es la notificación local. Es un aviso que se repite a una hora fija y que depende de datos que ya están en el celular.

5. Si el celular no tiene internet, ¿qué notificaciones siguen funcionando?

Siguen funcionando las notificaciones locales, porque las crea la propia app dentro del celular. Las push necesitan internet para llegar desde el servidor; si no hay conexión, se reciben cuando el celular vuelve a conectarse.

6. ¿Una app moderna puede funcionar solo con notificaciones locales?

Solo si todo lo que avisa ocurre dentro del celular, como alarmas o recordatorios. La mayoría de apps modernas necesita también las push, porque muchos avisos nacen fuera del dispositivo: mensajes, pagos o envíos. En TecsupNite la notificación es local porque el aviso aparece cuando la propia app termina de sincronizar la tienda.

10. Observaciones

1. La API de Fortnite no pide clave para la tienda ni para el detalle, así que no hubo que guardar ningún secreto en el proyecto.
2. Algunas entradas de la tienda no son cosméticos, como las pistas musicales y los autos, y hubo que filtrarlas antes de guardarlas.
3. Un mismo cosmético puede venderse solo y dentro de un pack; se muestra primero la versión suelta para que su precio sea el correcto.
4. El rol de cada usuario depende de que exista su documento en Firestore con el mismo código que tiene en Authentication; sin ese documento, el administrador entraba como usuario normal.
5. El emulador no tiene servicios de Google Play completos, pero Firebase Authentication y Firestore funcionaron igual para todas las pruebas.

11. Conclusiones

1. Tener el repositorio como interfaz permitió cambiar la API y Room por datos en memoria y probar el ViewModel sin emulador ni internet.
2. Usar Room como única fuente de verdad hizo que el modo sin conexión y los favoritos funcionen sin lógica extra en las pantallas.
3. Guardar el rol en Firestore, y no en la app, separa lo que ve cada persona de lo que realmente puede hacer, que lo deciden las reglas de seguridad.
4. Separar el motivo de un error de su texto ayuda a mostrar mensajes claros y a probarlos sin depender de Firebase.
5. Trabajar con commits pequeños en dos ramas dejó claro qué parte se hizo en la Fase 1 y qué se mejoró con IA en la Fase 2.